

SESIÓN 11 Spring Security básico (sin JWT todavía)

"Una API sin seguridad no es una API real."

Bloque 1 — Objetivos

Añadir spring-boot-starter-security

Entender **401 vs 403**

Configurar **SecurityFilterChain**

Proteger endpoints por rol

Permitir Swagger sin autenticación

Preparar terreno para JWT

Bloque 2 — Dependencia

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Bloque 3 — Estructura del proyecto

```
com.cladsb1.books
├── config
│   └── SecurityConfig
├── controller
├── service
├── repository
├── entity
├── dto
└── error
```



Bloque 4 — Teoría mínima imprescindible

Authentication

quién eres

Authorization

qué puedes hacer

401 Unauthorized

no estás autenticado

403 Forbidden

estás autenticado pero
no tienes permisos

Spring Security actúa como **filtro** antes del controller.



A) VERSIÓN CON LAMBDAS (estilo recomendado)



Bloque 5A — SecurityConfig CON lambdas



config/SecurityConfig.java (CON lambdas)

```
package com.cladsb1.books.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // En API REST típica (sin sesión/cookies) se desactiva CSRF
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                // Swagger abierto
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                // Ejemplo: permitimos búsquedas públicas
                .requestMatchers("/api/books/search").permitAll()
                // Todo lo demás de /api/books requiere ADMIN
                .requestMatchers("/api/books/**").hasRole("ADMIN")
                // Cualquier otro endpoint requiere estar autenticado
                .anyRequest().authenticated()
            )
            // Autenticación básica para demo
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        UserDetails admin = User.builder()
            .username("admin")
            .password(encoder.encode("admin123"))
            .roles("ADMIN")
            .build();

        UserDetails user = User.builder()
            .username("user")
            .password(encoder.encode("user123"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Bloque 6A — Seguridad por método CON lambdas

En el service (ejemplo):

```
import org.springframework.security.access.prepost.PreAuthorize;

@PreAuthorize("hasRole('ADMIN')")
public BookResponseDTO create(BookRequestDTO dto) {
    ...
}
```



B) VERSIÓN SIN LAMBDDAS (clases anónimas / sin DSL funcional)



Bloque 5B — SecurityConfig SIN lambdas



config/SecurityConfig.java (SIN lambdas)

```
package com.cladsb1.books.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        // 1) CSRF disable sin lambda (usando AbstractHttpConfigurer::disable como referencia estática
        // Si quieres 100% sin referencias, puedes usar Customizer con clase anónima (ver abajo).
        http.csrf(AbstractHttpConfigurer::disable);

        // 2) authorizeHttpRequests sin lambda: usando Customizer con clase anónima
        http.authorizeHttpRequests(new
Customizer<org.springframework.security.config.annotation.web.configurers.AuthorizeHttpRequestsConfigurer<HttpS
ecurity>.AuthorizationManagerRequestMatcherRegistry>() {
            @Override
            public void
customize(org.springframework.security.config.annotation.web.configurers.AuthorizeHttpRequestsConfigurer<HttpSe
curity>.AuthorizationManagerRequestMatcherRegistry auth) {
                auth.requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll();
                auth.requestMatchers("/api/books/search").permitAll();
                auth.requestMatchers("/api/books/**").hasRole("ADMIN");
                auth.anyRequest().authenticated();
            }
        });

        // 3) httpBasic sin lambda
        http.httpBasic(Customizer.withDefaults());

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        UserDetails admin = User.builder()
            .username("admin")
            .password(encoder.encode("admin123"))
            .roles("ADMIN")
            .build();

        UserDetails user = User.builder()
            .username("user")
            .password(encoder.encode("user123"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

- Esta versión es más "fea" porque el DSL moderno está pensado para lambdas.
- Pero sirve para quien no domina lambdas: se ve la idea sin sintaxis funcional.

Bloque 6B — Seguridad por método SIN lambdas

Las anotaciones @PreAuthorize("...") no usan lambdas, así que es igual:

```
@PreAuthorize("hasRole('ADMIN')")
public BookResponseDTO create(BookRequestDTO dto) {
    ...
}
```

Bloque 7 — Tabla de pruebas

Acción	Usuario	Resultado
Abrir Swagger	ninguno	✓ 200
GET /api/books/search	ninguno	✓ 200
POST /api/books	ninguno	✗ 401
POST /api/books	user/user123	✗ 403
POST /api/books	admin/admin123	✓ 201
DELETE /api/books/{id}	user/user123	✗ 403
DELETE /api/books/{id}	admin/admin123	✓ 204

Bloque 8 — Errores típicos

 No desactivar CSRF
→ POST/PUT/DELETE devuelven 403

 Poner
.hasRole("ROLE_ADMIN") (NO, se pone "ADMIN")

 Proteger Swagger sin querer → te quedas sin docs

 No diferenciar 401 vs 403 (muy común)

Bloque 9 — Cierre

 Mensaje clave:

"Spring Security es un filtro delante de tu API. Primero autenticación, luego autorización."